Handling Missing Data

General Principles

In many real-world datasets, some observations or parameters may be missing or partially known. In a standard frequentist approach, these missing values are often dropped, leading to biased estimates and a loss of information.

In the Bayesian framework, **missing data are simply treated as unknown parameters to be estimated within the model itself**. The entire joint distribution is evaluated, allowing the model to smoothly "impute" the missing values based on the observed data and prior relationships.

Considerations

> **Note**
>
> - **Parameters vs. Data**: In generative models, there is minimal mathematical distinction between "unobserved parameters" (like regression weights) and "unobserved data" (like missing survey responses). Both are simply unknowns governed by probability distributions.
> - **JAX Arrays are Immutable**: `BayesInference` uses JAX in the backend. JAX arrays do not support direct heterogeneous assignments (e.g., `y[ii_mis] = y_mis`). When splicing missing data into a larger array, you must use `.at[...].set(...)` operations.
> - **Partial Knowledge**: Sometimes covariance or correlation matrices are only partially known. Instead of discarding the known parts, we only sample the unknown cells uniformly within mathematically valid positive-definite bounds and reconstruct the matrix deterministically.

Example

Below are three typical missing data patterns translated into the **BayesInference (BI)** framework: 1. **Simple Missing Imputation**: Treating missing observations as a vector of latent variables. 2. **Partially Known Parameters**: Reconstructing a covariance matrix from known variances and an unknown covariance. 3. **Sliced Missing Data**: Inserting missing time-series steps cleanly into an array using JAX `at`.

Python

```python
from BI import bi
import jax.numpy as jnp

m = bi(platform="cpu")

# 1. Simple Missing Data Imputation ----------------------------
def missing_data_model(y_obs, N_mis):
    mu = m.dist.normal(0, 10, name="mu")
    sigma = m.dist.half_normal(10, name="sigma")

    # Treat missing data block as latent parameters
    with m.dist.plate("missing_obs", N_mis):
        y_mis = m.dist.normal(mu, sigma, name="y_mis")

    # Likelihood for OBSERVED data
    m.dist.normal(mu, sigma, obs=y_obs)

# 2. Partially Known Parameters (Covariance) --------------------
def partially_known_model(y, var1, var2):
    # Determine bounds based on positive-definite requirements
    max_cov = jnp.sqrt(var1 * var2)
    min_cov = -max_cov

    mu = m.dist.normal(0, 5, shape=(2,), name="mu")
    # Sample the unknown covariance within valid bounds
    cov = m.dist.uniform(min_cov, max_cov, name="cov")

    # Deterministically assemble the partially known matrix
    Sigma = jnp.array([
        [var1, cov],
        [cov,  var2]
```

```
    ])

    # Fit the Likelihood
    m.dist.multivariatenormal(loc=mu, covariance_matrix=Sigma, obs=y)

# 3. Sliced Missing Data (Time Series) -------------------------
def sliced_missing_model(y_obs, ii_obs, N_mis, ii_mis, N_total):
    sigma = m.dist.gamma(1, 1, name="sigma")

    # Missing points are parameters
    y_mis = m.dist.normal(0, 10, shape=(N_mis,), name="y_mis")

    # Assemble full array without in-place mutation using JAX .at
    y_full = jnp.zeros(N_total)
    y_full = y_full.at[ii_obs].set(y_obs)
    y_full = y_full.at[ii_mis].set(y_mis)

    # State-Space Time Series Likelihood: y[t] ~ normal(y[t-1], sigma)
    m.dist.normal(0, 100, obs=y_full[0])
    m.dist.normal(y_full[:-1], sigma, obs=y_full[1:])
```

Julia

```
using BayesianInference
using PythonCall

m = importBI(platform="cpu")
jnp = pyimport("jax.numpy")

# 1. Simple Missing Data Imputation ----------------------------
@BI function missing_data_model(y_obs, N_mis)
    mu = m.dist.normal(0, 10, name="mu")
    sigma = m.dist.half_normal(10, name="sigma")

    # Treat missing data block as latent parameters
    tmp = m.dist.plate("missing_obs", N_mis)
    m.dist.normal(mu, sigma, name="y_mis")
    # Note: In Python, `with plate:` is used. For Julia, follow the macro constraints or vect

    # Likelihood for OBSERVED data
    m.dist.normal(mu, sigma, obs=y_obs)
```

```
end

# 2. Partially Known Parameters (Covariance) -------------------
@BI function partially_known_model(y, var1, var2)
    max_cov = jnp.sqrt(var1 * var2)
    min_cov = -max_cov

    mu = m.dist.normal(0, 5, shape=(2,), name="mu")
    cov = m.dist.uniform(min_cov, max_cov, name="cov")

    # Assemble the matrix row-by-row
    row1 = jnp.stack([var1, cov])
    row2 = jnp.stack([cov, var2])
    Sigma = jnp.stack([row1, row2])

    # Fit the Likelihood
    m.dist.multivariatenormal(loc=mu, covariance_matrix=Sigma, obs=y)
end

# 3. Sliced Missing Data (Time Series) -------------------------
@BI function sliced_missing_model(y_obs, ii_obs, N_mis, ii_mis, N_total)
    sigma = m.dist.gamma(1, 1, name="sigma")
    y_mis = m.dist.normal(0, 10, shape=(N_mis,), name="y_mis")

    # Assemble full array using JAX immutable setters
    y_full = jnp.zeros(N_total)
    y_full = y_full.at[ii_obs].set(y_obs)
    y_full = y_full.at[ii_mis].set(y_mis)

    m.dist.normal(0, 100, obs=y_full[0])
    m.dist.normal(y_full[0:end-1], sigma, obs=y_full[1:end])
end
```

Mathematical Details

Bayesian Formulation

Missing data imputation assumes that the underlying generative process connects both the observed variables Y_{obs} and the missing variables Y_{mis}.

For a simple Gaussian imputation, the joint posterior of the parameters (μ, σ) and the missing

data Y_{mis} is proportional to:

$$P(\mu, \sigma, Y_{\text{mis}}|Y_{\text{obs}}) \propto P(Y_{\text{obs}}|\mu, \sigma)\, P(Y_{\text{mis}}|\mu, \sigma)\, P(\mu, \sigma)$$

Because both Y_{obs} and Y_{mis} are generated from the identical normal distribution Normal(μ, σ), the sampler naturally infers values for Y_{mis} that are highly plausible given the parameters μ, σ fitted to Y_{obs}.

For partially known covariance matrices, the reconstructed parameter Σ is mathematically bound by the positive-definite limits of a 2×2 matrix, leading to:

$$\det(\Sigma) > 0 \implies \text{var}_1\text{var}_2 - \text{cov}^2 > 0 \implies |\text{cov}| < \sqrt{\text{var}_1\text{var}_2}$$

Notes

> **Note**
>
> - **Missing Completely At Random (MCAR) vs Not Missing At Random (NMAR)**: The structural examples shown above effectively operate under the assumption that the probability of data being missing is independent of the value itself (MCAR or MAR). If the missingness mechanism depends on the missing value itself (NMAR)—e.g., people with low income refusing to report their income—a secondary likelihood must explicitly model the probability of the data being missing (`is_missing ~ Bernoulli(p)`).

Reference(s)

1. Stan User's Guide: Missing Data